ESPECIFICACIÓN TÉCNICA Y DISEÑO FRONTEND

Proyecto Grupo 8: Aplicación de Aplicación Asistencia Institucional (Konrad Check)

link al github:
https://github.com/ValerosoV/KonradCheck

Integranal Equipo:	Juan David Valero Vanegas, Stiven Adolfo Rodriguez Martinez, Juan Kamilo Chacon Arias Docente
Fecha de Entrega:	22 de Octubre, 2026
Versión del Documento:	2.0.0
Asignatura:	Desarrollo de Aplicaciones Móviles

1. Descripción General del Proyecto

El presente proyecto consiste en el desarrollo de una aplicación móvil de carácter institucional orientada al registro de la toma de asistencia estudiantil durante las clases presenciales y las reuniones obligatorias, logrando esto mediante el uso de la tecnología Quick Response code (QR code) utilizando sus matrices dentro de la app para distribuir los datos únicos de registro de cada respectivo estudiante que sea signado como asistente.

2. Problemática o Necesidad Identificada

Dentro la institución de educación superior Fundación Universitaria Konrad Lorenz se tienen entre sus eventos académicos la realización de las clases y las reuniones obligatorias informativas, y para dichos eventos se busca garantizar que la información recopilada de su asistencia sea lo más confiable posible, en el caso de las clases su toma de asistencia depende de dedicar un lapso de tiempo exclusivamente para realizar un llamado a lista, proceso donde varios docentes dependen de un Excel que tendrán como soporte donde se registra la información y se pueden cometer errores de digitación, mientras que en la reuniones se maneja una digitación dentro de un formulario web de Microsoft distribuido a través de un QR único mostrado en la presentación proyectada, formulario accesible desde cualquiera lugar y por cualquier persona que tenga el QR, aun pese a no asistir a la reunión de manera presencial o virtual.

3. Objetivos del Proyecto

3.1 Objetivo General

Desarrollar una aplicación móvil institucional que automatice y valide el registro de asistencia de los estudiantes a clases y reuniones de la Fundación Universitaria Konrad Lorenz, mediante la generación y validación de códigos QR únicos por estudiante y dinámicos, garantizando la veracidad de la información y la optimización del tiempo.

3.2 Objetivos Específicos

- Diseñar una interfaz de usuario (UI) intuitiva y ágil que permita a los estudiantes y docentes interactuar con la aplicación sin fricciones.
- Implementar un sistema de autenticación seguro que valide las credenciales institucionales de los usuarios.
- Desarrollar un módulo de generación de códigos QR para evitar la suplantación mediante envío de enlaces.
- Crear una base de datos centralizada para almacenar y actualizar los registros de asistencia en tiempo real que se pueda conectar
- Estructurar un módulo de consulta donde los docentes puedan visualizar los reportes de asistencia de sus respectivas asignaturas o eventos.

4. Requerimientos Funcionales y No Funcionales (Frontend)

4.1 Requerimientos Funcionales (RF)

ID	Requerimiento Funcional	Descripción
RF-01	INICIO DE SESIÓN	Al pulsar «Iniciar sesión» tras ingresar un correo y contraseña validas se redirige a la pantalla de reuniones del tipo correspondiente de usuario.
RF-02	CREAR REUNIÓN	Muestra un formulario para ingresar los datos de una reunión y cargar una plantilla que enliste los estudiantes que asistan
RF-03	MOSTRAR QR	Tras presionar «QR» en la interfaz que enlista las clases, se redireccionara a la pantalla respectiva con el código QR del estudiante junto con el nombre, el ID y la materia del estudiante.
RF-04	ESCANEAR QR	Presenta una pantalla donde se abre la cámara para realizar el escaneo de un QR e ingresar al sistema la información de Check-in para el estudiante
RF-05	CONSULTAR ASISTENCIA PERSONAL	Muestra un resumen del porcentaje de asistencia del Usuario Estudiante y una lista de los registros de la asistencia por evento, implementa filtros según la clase correspondiente que desee consultar.
RF-06	VISUALIZAR PERFIL	Muestra los datos del usuario, muestra configuraciones futuras y tiene un botón para cerrar la sesión de usuario.
RF-07	CONSULTAR ASISTENCIA A REUNIÓN	Presenta un resumen y los estados de asistencia para una reunión, con una barra de búsqueda para buscar estudiantes particulares y exportar el

		reporte del evento mostrado.
RF-08	RECUPERAR CONTRASEÑA	

4.2 Requerimientos No Funcionales (RNF)

ID	Categoría	Especificación Técnica
RNF-01	Rendimiento	Tiempo de carga de pantalla inferior a 1.5s
RNF-02	Compatibilidad	Funcionamiento en Android e iOS.
RNF-03	Seguridad	Almacenamiento seguro de tokens de acceso
RNF-04	Disponibilidad	Servidor operativo durante todas las franjas de clases.

5. Inventario de Pantallas de la Aplicación

A continuación, se detalla el inventario sistemático de todas las vistas desarrolladas dentro del ecosistema frontend:

Código	Nombre de Pantalla	Descripción Resumida
SCR-01	Login	Autentica a estudiantes, docentes y administrativos con sus credenciales institucionales y los dirige a las funciones correspondientes a su rol.
SCR-02	Recuperar contraseña	Permite solicitar la recuperación de la cuenta mediante el correo institucional y recibir instrucciones para establecer una nueva contraseña.

SCR-03	Reuniones	Presenta las reuniones y clases del estudiante con horarios, ubicaciones y estado de asistencia, y permite acceder al QR de cada sesión.
SCR-04	Mi QR	Genera y muestra el código QR personal asociado a una reunión para que el estudiante registre su asistencia.
SCR-05	Historial	Permite consultar el resumen y los registros de asistencia del estudiante, con opciones para revisar la información por asignatura. |
SCR-06	Perfil	Muestra y permite gestionar la información de la cuenta, las preferencias, las opciones de seguridad y el cierre de sesión.
SCR-07	ReunionesPersonal	Lista las reuniones gestionadas por el usuario, muestra sus detalles y asistencia registrada, y permite consultar reportes o programar eventos.
SCR-08	Programar reunión	Permite crear una reunión indicando nombre, tipo, ubicación, descripción, fecha, horario y listado esperado de asistentes.
SCR-09	Escanear QR	Permite escanear el código QR de un estudiante, validar su identidad y la reunión activa, y registrar su asistencia.

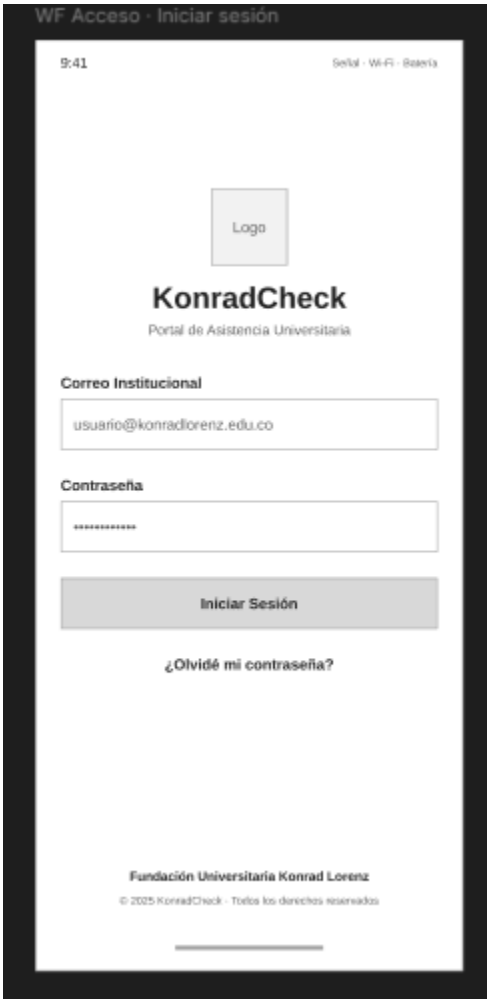
SCR-10	Reporte de asistencia	Presenta el registro de asistencia de una reunión, permite buscar estudiantes y exportar el reporte.

6. Wireframes de las Pantallas

Los wireframes estructuran el layout conceptual de la interfaz sin distracción visual de colores o imágenes.

Wireframes de Baja Fidelidad

SCR-01: Login



SCR-02: RecuperarContraseña



SCR-03: Reuniones



RF-04: MiQR


WF Estudiante · Registrar asistencia QR
9:41
Señal · Wi-Fi · Batería
Sistemas Operativos
Registrar Asistencia
Código QR
Santiago Rincón
ID: 22019483
El código se actualiza en 24s
Muestra este código al docente
El código se regenera automáticamente para evitar fraudes.
Inicio
Historial
Perfil

RF-05: Historial



RF-06: Perfil


WF Estudiante · Perfil y configuración
9:41
Señal · Wi-Fi · Batería
SR
Santiago Rincón
santiago.rincon@konradlorenz.edu.co
Estudiante Pregrado
Configuración General
Notificaciones de Asistencia
Activadas ›
Cambiar Contraseña
›
Acerca de la Aplicación
v1.4.2 ›
Soporte de Tecnología
›
Cerrar Sesión Activa
Inicio
Mi QR
Historial
Perfil

RF-07: ReunionesPersonal



SCR-08: ProgramarReunión



SCR-09: Escanear QR



SCR-10: Reporte de asistencia



7. Mockups de Alta Fidelidad

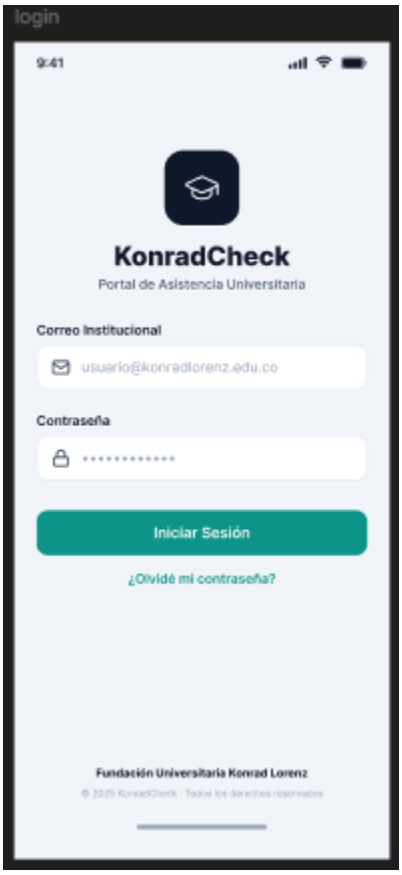
Los mockups incorporan el estilo visual final, paleta cromática, tipografías e iconografía oficial.

> **Mockups Visuales**
> *[Espacio reservado para Mockups visuales de alta fidelidad con acabado final UI]*
> *Contiene los layouts finales exportados desde herramientas de diseño (Figma / Adobe XD).*

8. Diseño UX/UI

El sistema de diseño define las directrices estéticas y funcionales para asegurar coherencia en la aplicación:

Interfaz 1: Logins



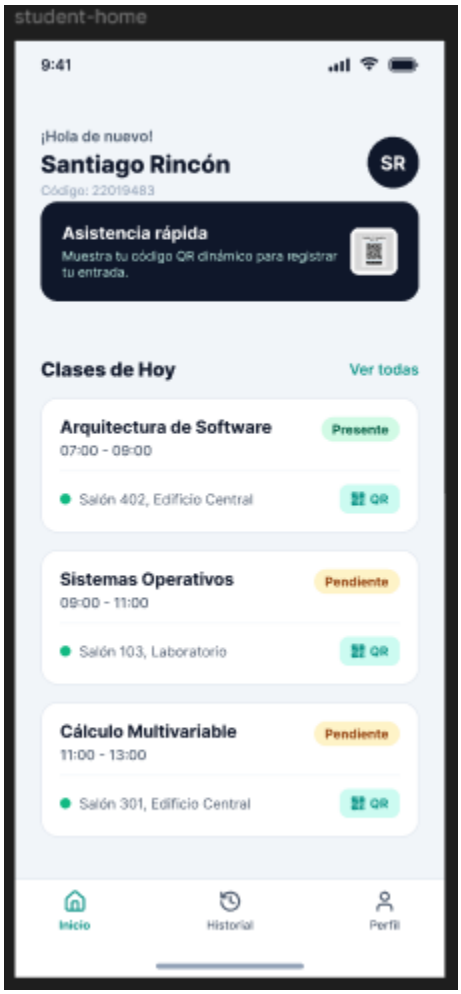
Descripción:
Se trata de la entrada de la aplicación, la cual validará las credenciales del Usuario, a futuro distinguirá los diversos tipos de usuarios. Utiliza los campos de correo, contraseña y su botón de inicio de sesión para redireccionar a la interfaz por defecto del respectivo tipo de usuario en caso de contar con un acceso valido, adicionalmente cuenta con un texto presionarle para dirigirse a la recuperación de contraseñas, y un acceso provisional a las interfaces del docente/administrativo.

Interfaz 2: Recuperar contraseña



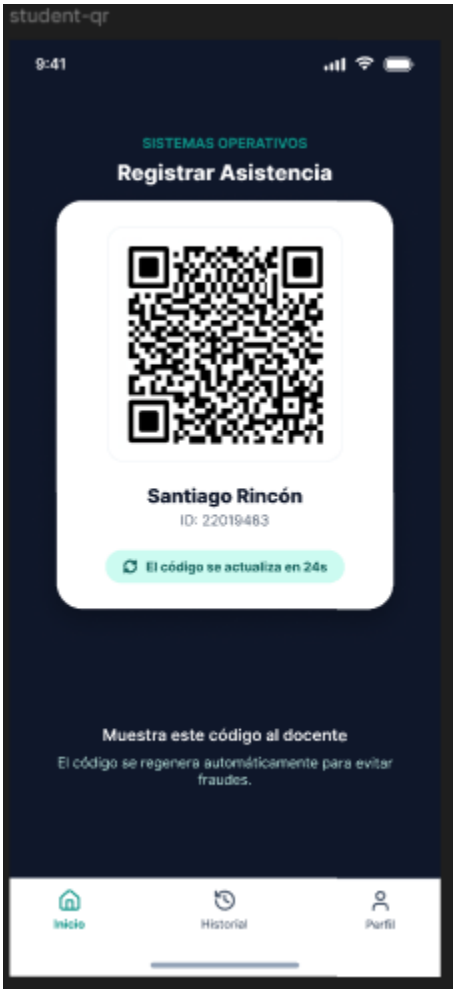
Descripción:
permite ingresar el correo institucional para solicitar instrucciones de recuperación y volver al Login.

Interfaz 3: Reuniones



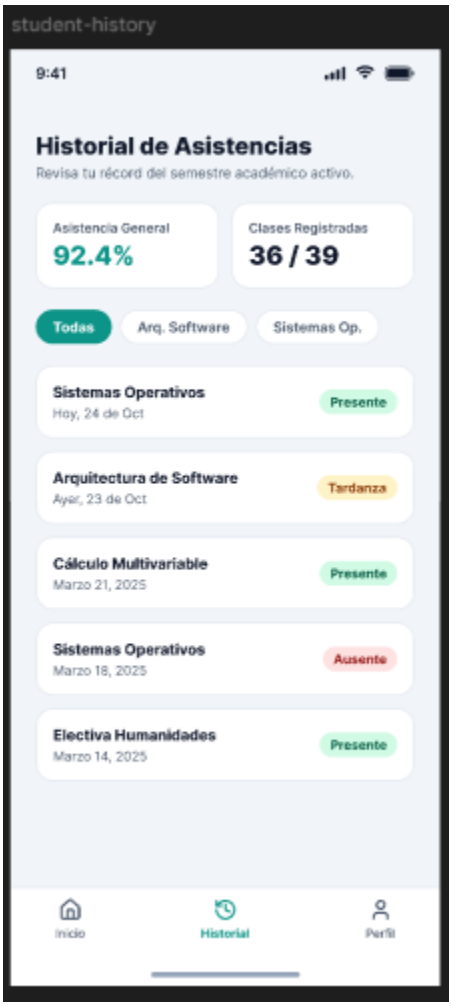
Descripción:
Esta interfaz es la predeterminada al ingresar a la APP utilizando una cuenta de estudiante, en la parte superior muestran unos pocos datos del estudiante para corroborar la identidad del dueño de la cuenta al acceder, y su función principal es enlistar las clases de día vigente, ofreciendo el título de la asignatura, la hora inicial y final de la reunión, el salón donde trascurrirá, un aviso del asignado referente a su asistencia a la clase, y botón para mostrar el QR asignado al estudiante para registrar su asistencia.

Interfaz 4: Mi QR



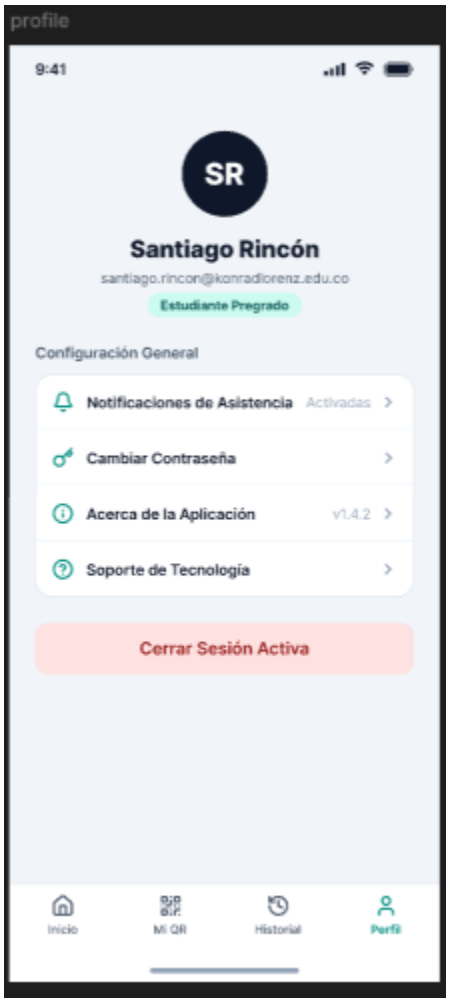
Descripción:
En esta interfaz se mostrará el QR único del estudiante junto a unos datos personales para corroborar la validez de su asistencia.

Interfaz 5: Historial



Descripción:
El históricos de asistencias del estudiante posee métricas para la autoevaluación personal de la asistencia del estudiante a sus clases/reuniones asignadas, y un listado de las clases denotando el resultado de cada instancia, listado capaz de aplicársele filtros por asignaturas para facilitar el monitoreo personal de la asistencia.

Interfaz 6: Perfil



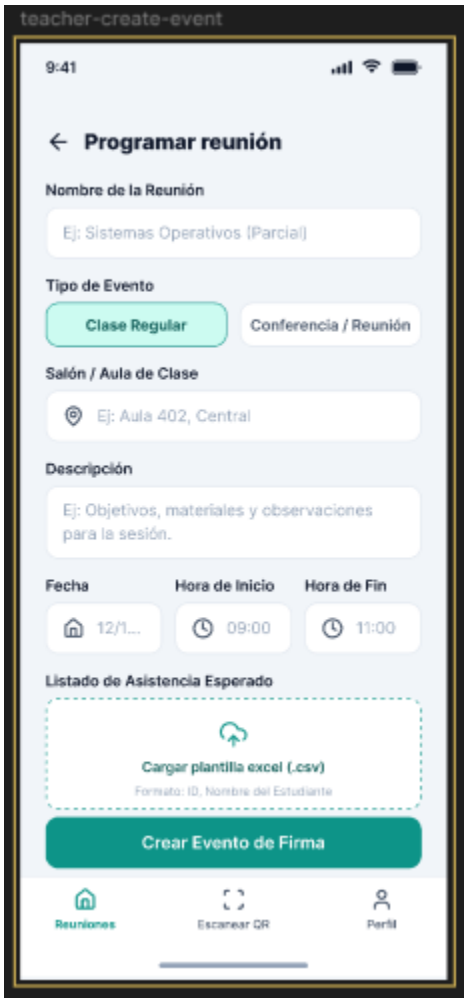
Descripción:
Una interfaz cuya principal función tener la posibilidad de cerrar sesión, y adicionalmente el mostrar configuraciones futuras para el uso de la app.

Interfaz 7: Reuniones Personal



Descripción:
Esta interfaz es la vista por defecto para un usuario de la app perteneciente al personal de la institución (Docentes o Administrativos), en ella se ofrece una opción para acceder al formulario de adición de reuniones, y un listado de las reuniones de las que será participe, ofrecientes la opción de consultar la asistencia registrada de los estudiantes.

Interfaz 8: Programar reunión



Descripción:

En este espacio se muestran los campos del formulario para programar una reunión de la institución, consta de un campo para el titulo/nombre de la reunión, la naturaleza del evento por venir, la ubicación, la fecha, las horas de inicio y fin, más una Dropbox para cargar un archivo compatible que registrara los estudiantes asociados a la reunión.

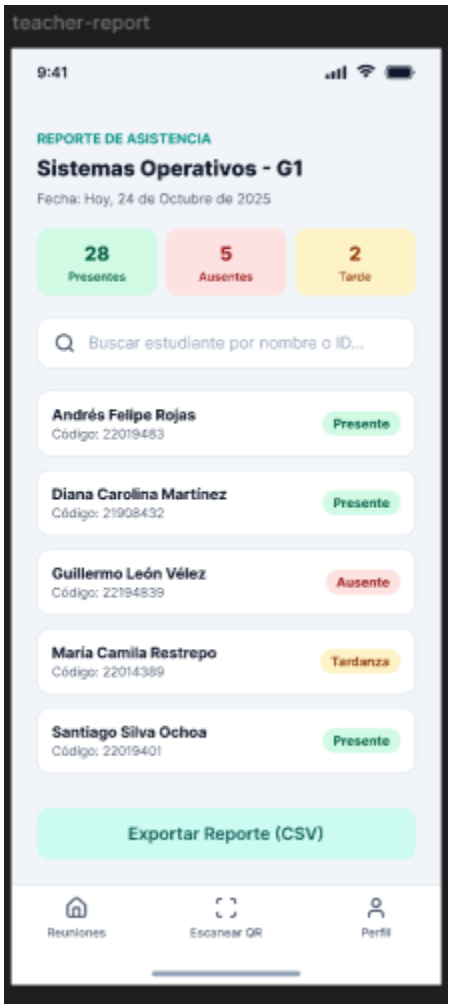
Interfaz 9: Escanear QR



Descripción:

En esta interfaz se mostrará la imagen de la cámara para ajustar su escáner que reconocerá los QRs pertinentes de los estudiantes que asistan a una reunión.

Interfaz 10: Reportes de asistencia



Descripción:

Por último, se tiene la interfaz que corresponde al reporte de asistencia de una instancia de clase/reunión institucional, mostrando detalles de la reunión, la cantidad de personas en cada categoría de asistencia, una searchbar para consultar estudiantes en particular que requiera el docente/administrador, un listado de los asistentes registrados y finaliza con una opción para exportar el reporte presente.

9. Diagrama de Navegación

El diagrama de navegación define el flujo e interacción entre pantallas mediante React Navigation

> **Esquema de Flujo de Navegación**
> *Flujo General:*
> *[Inicio de la aplicación]*

```
  |
[Login] ————————————————————> [Recuperar contraseña]
  |
  | Inicio de sesión
  | (en la versión final, según el rol del usuario)
  |
  +——> [Área de estudiante]
  |      |
  |      +—— Tab: [Reuniones] ——> [Mi QR]
  |      +—— Tab: [Historial de asistencia]
  |      +—— Tab: [Perfil]
  |
  +——> [Área de personal]
         |
         +—— Tab: [Reuniones]
         |      +——> [Programar reunión]
         |      +——> [Reportes de asistencia]
         |
         +—— Tab: [Escanear QR]
         +—— Tab: [Perfil]
```

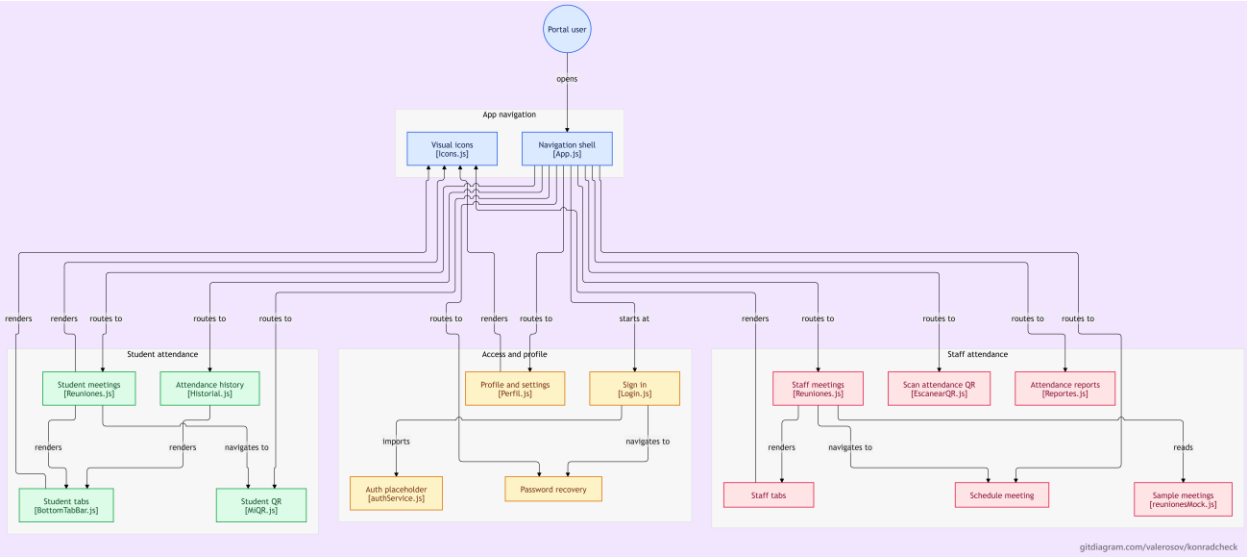
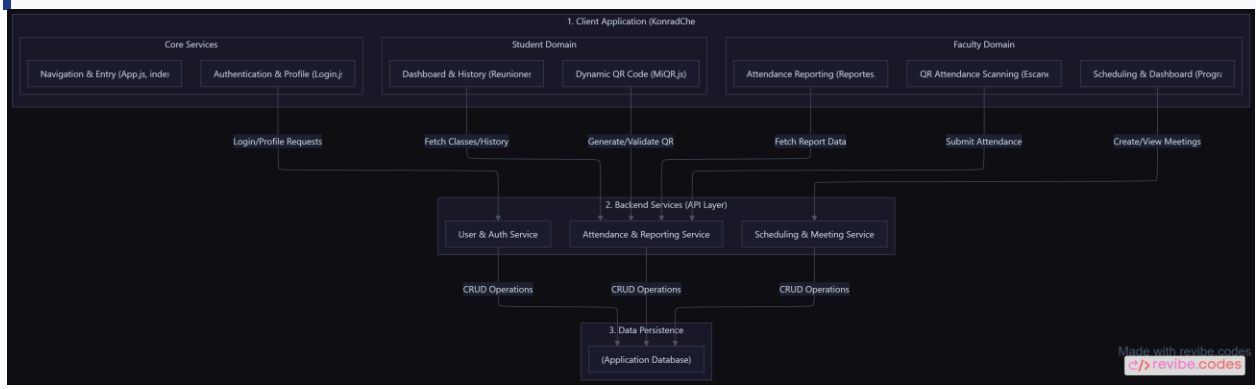


Imagen generada con: https://gitdiagram.com/

10. Diagrama de Arquitectura

KonradCheck se plantea como una aplicación móvil de tres capas: cliente, servicios de backend y persistencia de datos. Esta arquitectura separa la interfaz que utiliza cada persona de las reglas del sistema y del almacenamiento permanente. Así, la aplicación móvil se ocupa de presentar la información y enviar solicitudes; el backend valida esas solicitudes y ejecuta las operaciones; y la base de datos conserva usuarios, reuniones y registros de asistencia.

> **Arquitectura Frontend**
> *Aplicación móvil*
> |
> | *Solicitudes y respuestas mediante una API*
> *v*
> *Servicios de backend*
> |
> | *Consultas y operaciones de datos*
> *v*
> *Base de datos*



-Capa de cliente móvil: La aplicación está construida con Expo, React Native y JavaScript. El archivo `App.js` centraliza actualmente la navegación y registra las rutas principales, mientras que `index.js` actúa como punto de entrada de Expo. Las pantallas se agrupan según el estado de sesión y el perfil del usuario: las funciones del estudiante incluyen reuniones, código QR e historial; las del personal incluyen reuniones, programación, escaneo de QR y reportes. Esta organización por áreas permite que cada perfil tenga acceso a los flujos correspondientes.

En una versión final, el inicio de sesión consultaría al backend y, una vez validada la identidad, la aplicación utilizaría el rol recibido para dirigir al usuario al área de estudiante o de personal. Las pantallas no deberían encargarse directamente de acceder a la base de datos: enviarían solicitudes a través de una capa cliente de API, que podría organizarse en módulos por dominio, como autenticación, reuniones y asistencia. Por ejemplo, las vistas de reuniones solicitarían los datos al servicio correspondiente y presentarían en pantalla la respuesta recibida.

-Capa de servicios de backend: El backend expondría una API para conectar la aplicación con las reglas del negocio. Según el diagrama, podría organizarse en servicios de usuarios y autenticación, programación de reuniones, y asistencia y reportes. El servicio de autenticación validaría las credenciales y los permisos; el de reuniones permitiría consultar, crear o modificar reuniones; y el de asistencia procesaría los escaneos de códigos QR y ofrecería los datos necesarios para generar reportes.

En el flujo de asistencia, la aplicación del personal enviaría al backend la información obtenida al escanear el código y la reunión asociada. El servidor comprobaría que el código corresponde a un usuario válido y que la reunión permite registrar asistencia antes de guardar el resultado. Para evitar registros incorrectos o duplicados, estas validaciones y la decisión final deberían realizarse en el backend, no únicamente en la interfaz móvil.

-Capa de persistencia: La base de datos almacenaría de forma permanente la información de usuarios, roles, reuniones y asistencias. Los servicios del backend realizarían las operaciones de consulta y actualización necesarias, manteniendo las reglas de acceso entre perfiles. Por ejemplo, un estudiante podría consultar su historial, mientras que el personal autorizado podría programar reuniones y consultar reportes de asistencia.

El código actual ya aporta la estructura inicial del cliente: contiene las pantallas, las rutas de navegación y una organización separada para estudiantes y personal. Sin embargo, `authService.js` todavía describe una integración futura y los datos de reuniones se encuentran en `reunionesMock.js`. Por tanto, el backend, la API y la persistencia descritos aquí corresponden a la arquitectura ideal propuesta, no a componentes que ya estén implementados.

11. Estructura de Carpetas del Proyecto

La organización del código fuente en React Native sigue el estándar de modularización limpia:

Árbol de Directorios del Código Fuente

```
KonradCheck/
├── .gitignore              # Archivos excluidos del control de versiones
├── README.md               # Descripción general del repositorio
├── notas.txt               # Archivo de notas del proyecto
└── KonradCheckApp/
    ├── App.js              # Configuración de navegación y registro de pantallas
```

```
├── index.js              # Punto de entrada de Expo
├── app.json              # Configuración de la aplicación Expo
├── package.json          # Dependencias y comandos del proyecto
├── package-lock.json     # Versiones exactas de las dependencias
├── assets/               # Íconos e imágenes de la aplicación
│   ├── android-icon-background.png
│   ├── android-icon-foreground.png
│   ├── android-icon-monochrome.png
│   ├── favicon.png
│   ├── icon.png
│   └── splash-icon.png
├── data/
│   └── reunionesMock.js  # Datos de ejemplo para las reuniones
├── screens/              # Pantallas y componentes de interfaz
│   ├── components/
│   │   └── Icons.js      # Íconos reutilizables
│   ├── noSesion/
│   │   ├── Login.js      # Inicio de sesión
│   │   └── RecuperarContrasena.js
│   └── conSesion/
│       ├── Perfil.js     # Perfil del usuario
│       ├── Estudiante/
│       │   ├── BottomTabBar.js
│       │   ├── Historial.js
│       │   ├── MiQR.js
│       │   └── Reuniones.js
│       └── Personal/
│           ├── BottomTabBarPersonal.js
│           ├── EscanearQR.js
│           ├── ProgramarReunion.js
│           ├── Reportes.js
│           └── Reuniones.js
└── services/
    └── authService.js    # Módulo previsto para la lógica de autenticación
```

No se incluyeron «node_modules/», «.expo/» ni « dist/» porque son dependencias o archivos generados, no parte del código fuente principal.

12. Descripción de la Implementación Inicial en React Native

El proyecto **KonradCheck** está desarrollado con **Expo, React Native y JavaScript**, tecnologías que permiten crear una aplicación móvil con una base de código común para ejecutarla en diferentes plataformas. Expo facilita la configuración y ejecución del proyecto, mientras que React Native permite construir la interfaz mediante componentes adaptables a dispositivos móviles.

La aplicación está organizada en módulos que separan la navegación, las pantallas, los recursos visuales, los datos de prueba y los servicios. El archivo `index.js` funciona como punto de entrada y registra la aplicación en Expo. Desde `App.js` se configura la navegación mediante React Navigation y se registran las pantallas principales, como inicio de sesión, recuperación de contraseña, reuniones, perfil, historial, escaneo de QR y reportes.

Las pantallas están agrupadas según el estado de sesión y el tipo de usuario. En `screens/noSesion/` se encuentran las vistas disponibles antes de iniciar sesión. En `screens/conSesion/Estudiante/` se organizan las funciones dirigidas al estudiante, como consultar reuniones, mostrar su código QR y revisar su historial. Por su parte, `screens/conSesion/Personal/` contiene las vistas orientadas al personal, como consultar y programar reuniones, escanear códigos QR y revisar reportes de asistencia. También se incluyen componentes visuales reutilizables, como los íconos y las barras de navegación.

El proyecto cuenta además con una carpeta `assets/` para almacenar los íconos e imágenes, una carpeta `data/` con datos de ejemplo para las reuniones y una carpeta `services/` destinada a centralizar operaciones de servicios, como la autenticación. Esta organización permite mantener cada parte de la aplicación en un lugar definido y facilita ampliar sus funcionalidades.

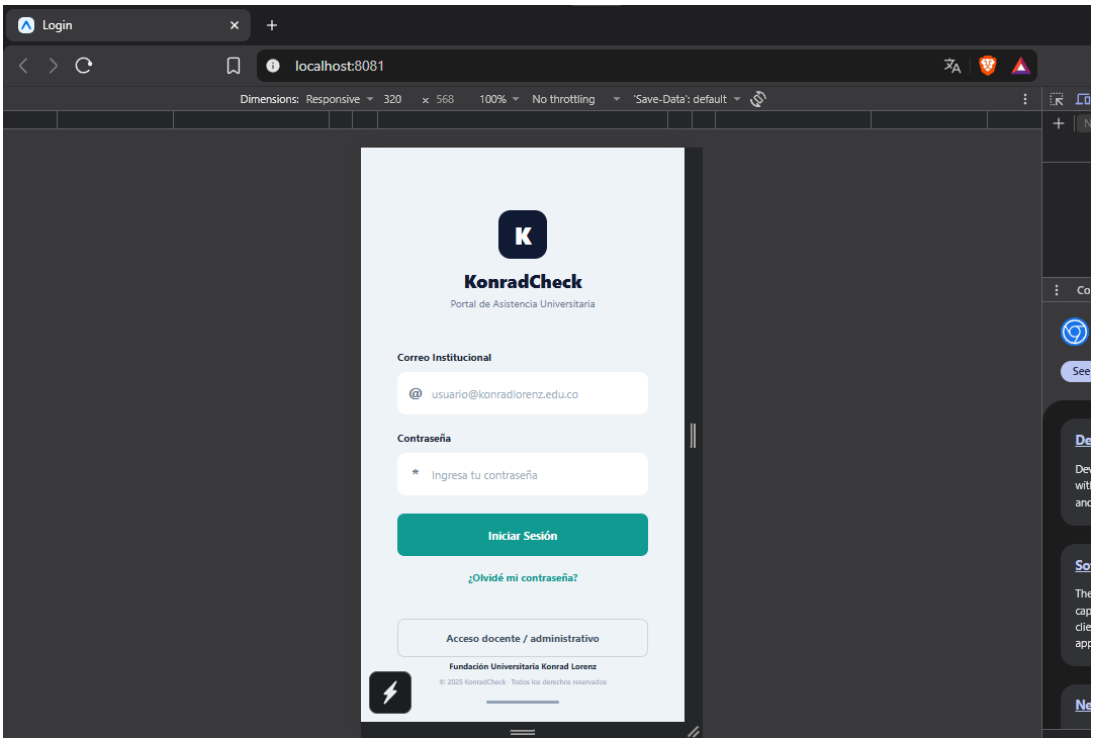
Actualmente, la aplicación presenta principalmente la estructura de interfaz y navegación. Algunas pantallas utilizan datos de muestra y ciertas acciones, como la autenticación, el escaneo real de códigos QR o la persistencia de reuniones y reportes, todavía requieren conectarse con un Backend para funcionar de manera completa.

13. Evidencias de Ejecución de Pantallas Desarrolladas

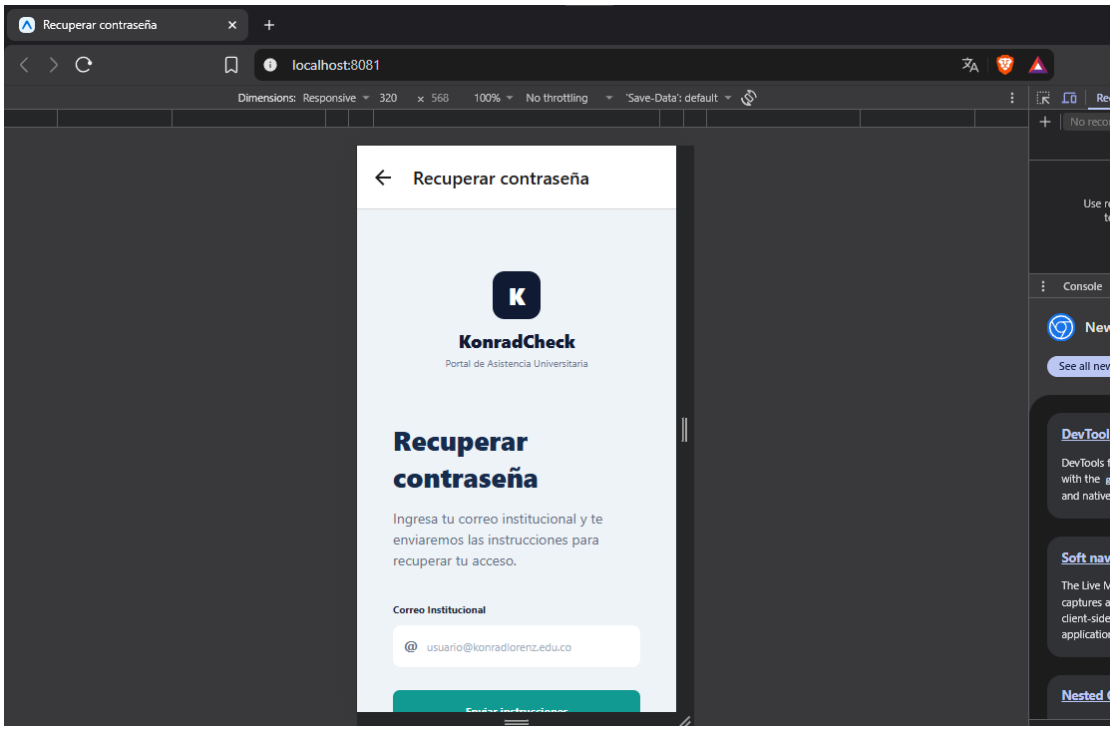
Se presentan las capturas de pantalla de la ejecución en VisualStudioCode

Evidencias de Ejecución

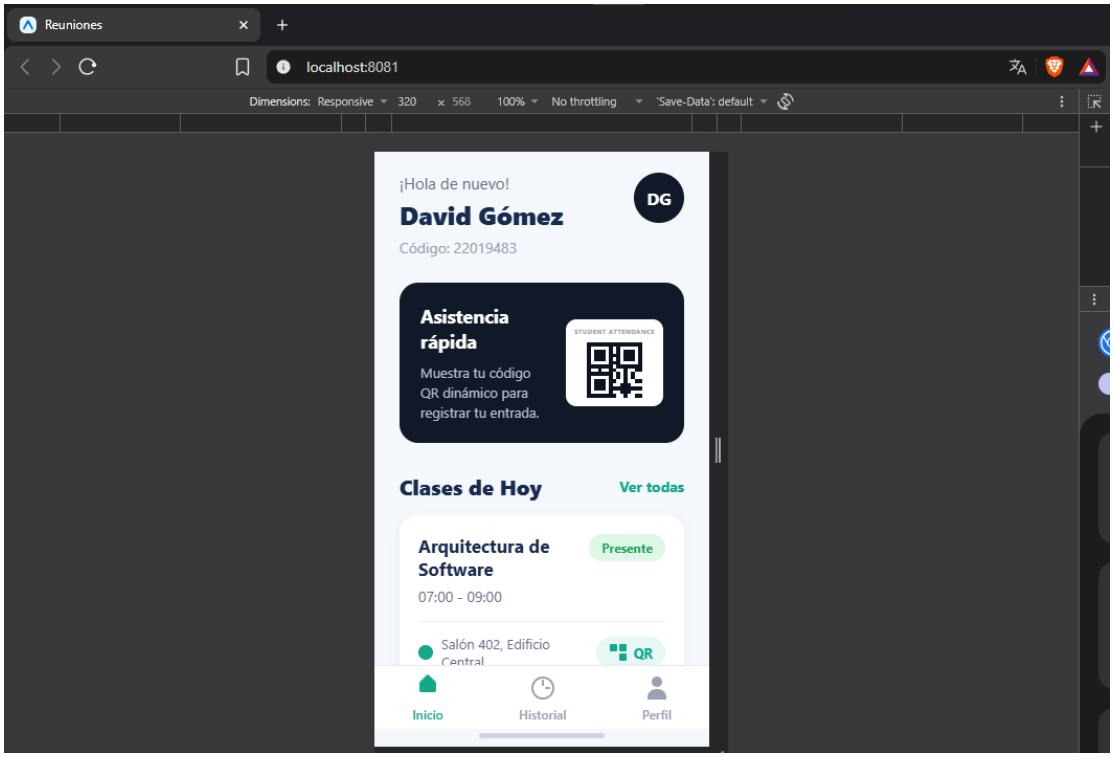
[Captura de pantalla: Ejecución de SCR-01: Login]



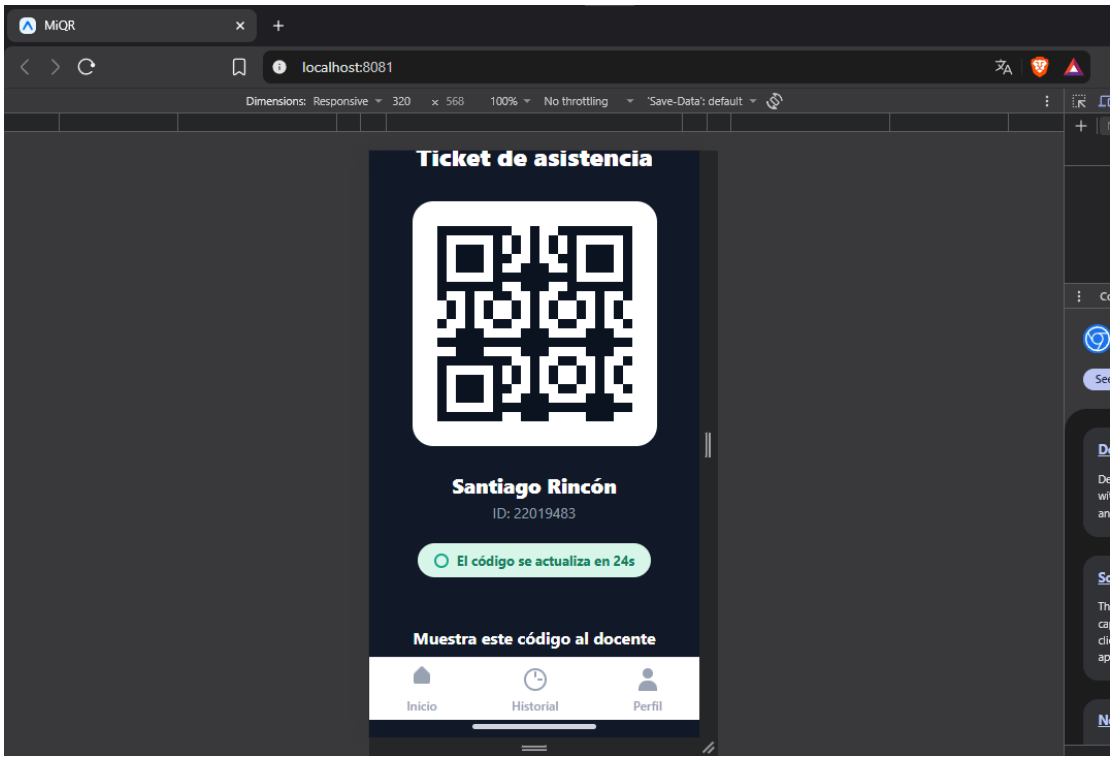
[Captura de pantalla: Ejecución de SCR-02: RecuperarContraseña]



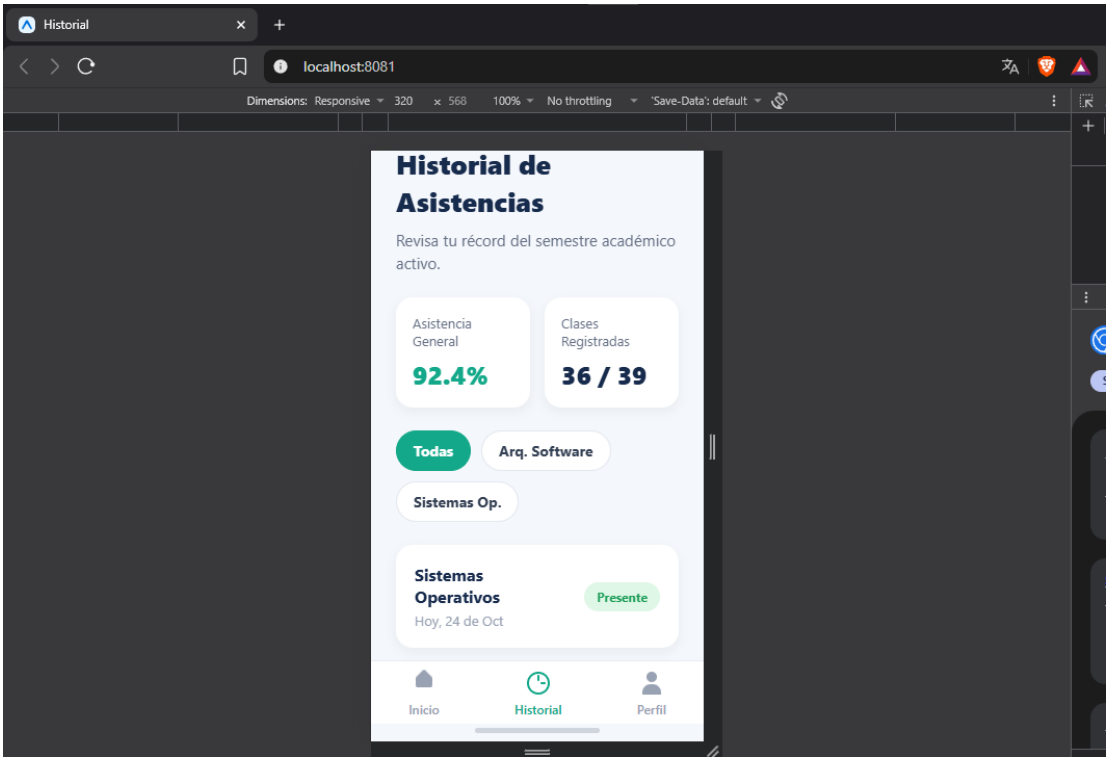
[Captura de pantalla: Ejecución de SCR-03: Reuniones]



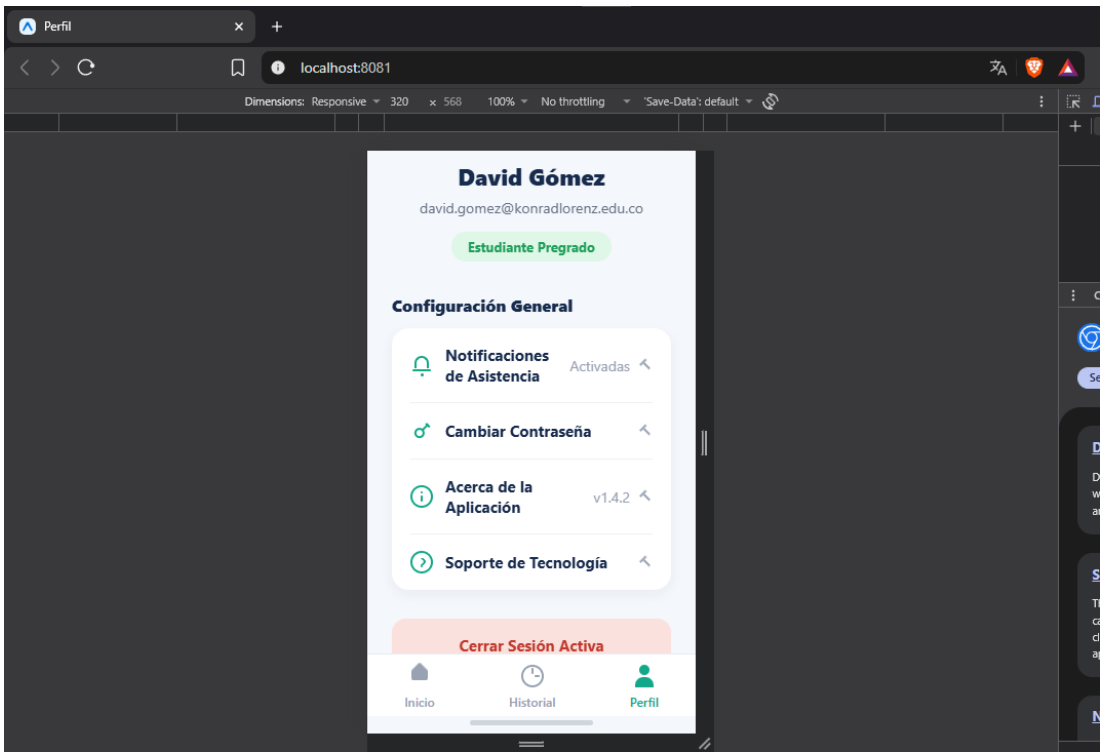
[Captura de pantalla: Ejecución de SCR-04: MiQR]



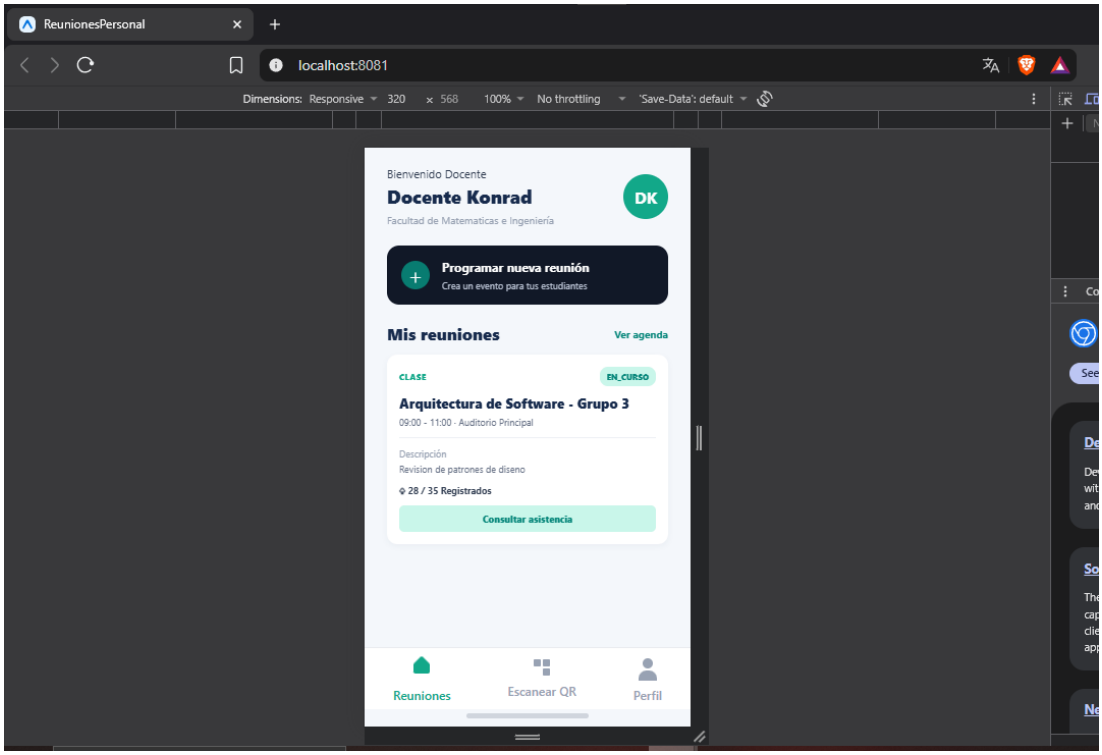
[Captura de pantalla: Ejecución de SCR-05: Historial]



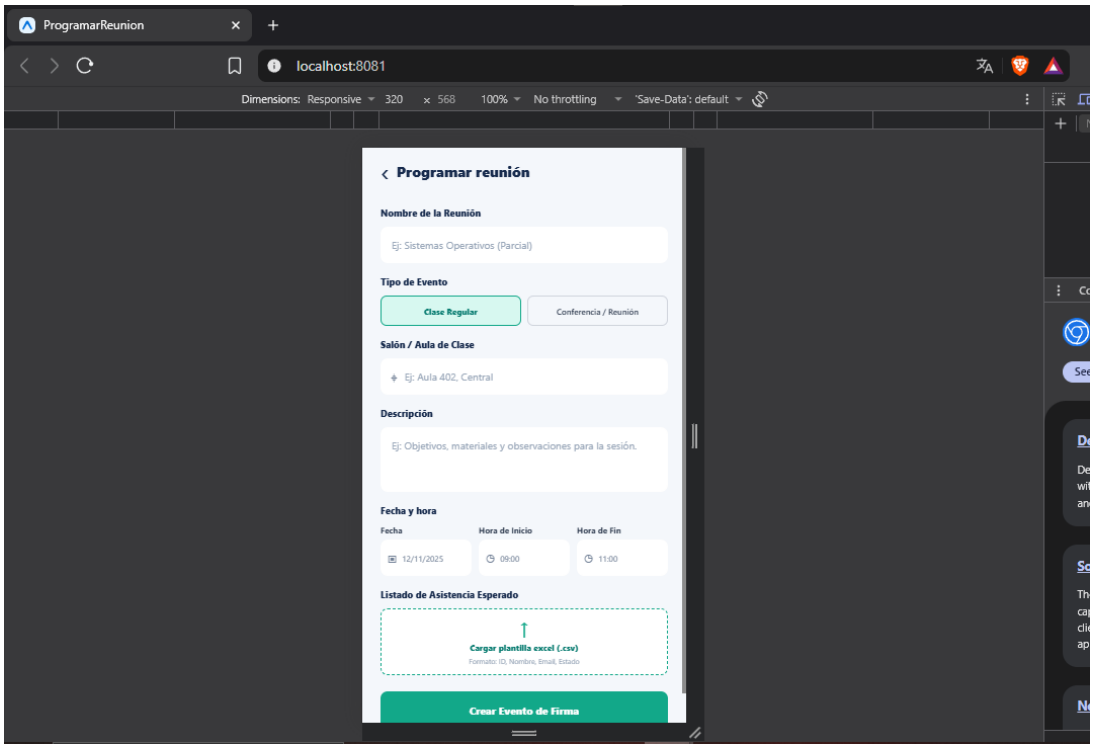
[Captura de pantalla: Ejecución de SCR-06: Perfil]



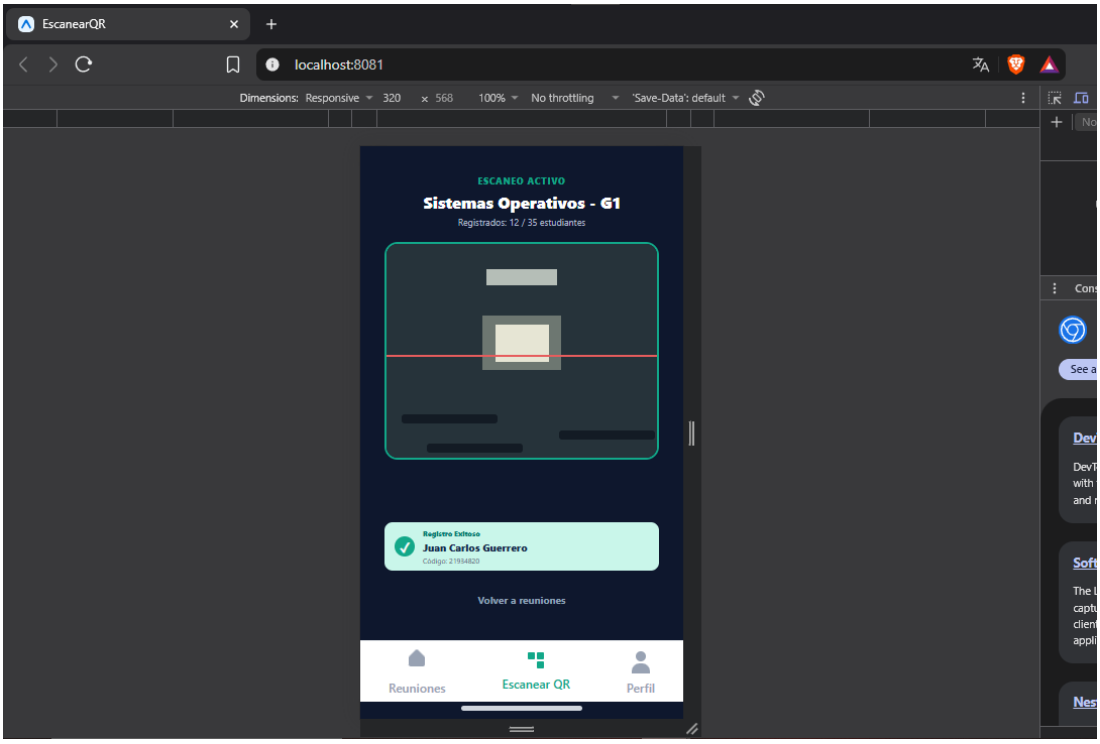
[Captura de pantalla: Ejecución de SCR-07: ReunionesPersonal]



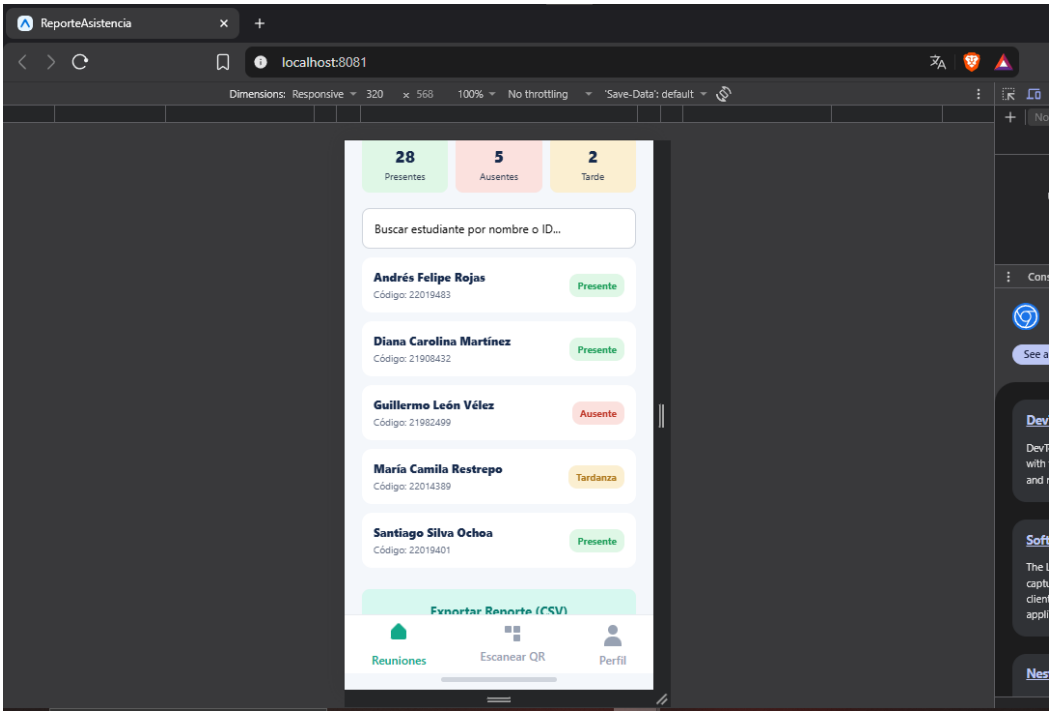
[Captura de pantalla: Ejecución de SCR-08: ProgramarReunion]



[Captura de pantalla: Ejecución de SCR-09: EscanearQR]



[Captura de pantalla: Ejecución de SCR-10: ReporteAsistencias]



14. Instrucciones para Instalar y Ejecutar el Frontend

Siga estos pasos para preparar y ejecutar KonradCheck en un entorno local. La aplicación está desarrollada con React Native y Expo, y se encuentra en la carpeta `KonradCheckApp` del repositorio.

14.1 Requisitos Previos

- Node.js o superior.
  - npm, incluido con Node.js.
  - Git, si se va a clonar el repositorio.
  - Un editor de código, como Visual Studio Code.
  - Para Android: un emulador configurado en Android Studio o un dispositivo móvil con Expo Go compatible.
  - Para iOS Simulator: un equipo con macOS y Xcode. También se puede usar un dispositivo iOS con Expo Go compatible.

14.2 Pasos de Ejecución

Comandos de Consola

1. Clonar el repositorio
git clone https://github.com/ValerosoV/KonradCheck
cd proyecto-frontend-rn

2. Instalar dependenciasE
npm install

3. Instalación de pods (Solo para iOS en macOS)
cd ios && pod install && cd ..

4. Ejecutar en una plataforma específica

En una nueva terminal, ubicada también en la carpeta KonradCheckApp, ejecute el comando correspondiente:

npm run android

Para Android, use un emulador configurado en Android Studio o un dispositivo con Expo Go compatible.

npm run ios

Para iOS Simulator se requiere macOS y Xcode. También puede usar un dispositivo iOS con Expo

> *Go compatible.*
>
> ***npm run web***
>
> *Para iniciar la versión web en el navegador.*
>
> *Notas: Ejecute los comandos desde la carpeta KonradCheckApp, donde se encuentra package.json. No es necesario ejecutar pod install para estos pasos de Expo. Para detener el servidor, presione Ctrl + C en la terminal.*